TECHNICAL SPECIFICATION · FINAL PROJECT REPORT

RoadMind

Explainable Predictive AI Co-Pilot for Advanced Driver Assistance Systems (ADAS)

A working, end-to-end implementation: a machine-learning pipeline trained on real US accident data, SHAP-based explainability, a FastAPI backend, and a production-styled Next.js dashboard — with real, honestly-reported results throughout.

Prepared by	[Student Name]
Roll No. / Course	[Roll Number / Course Code]
Submitted to	[Faculty / Advisor Name]
Institution	[Institution Name]
Date	[Submission Date]
Dataset	US Accidents (2016–2023), Kaggle, CC BY-NC-SA 4.0

Table of Contents

Note: *Word displays a placeholder until this field is refreshed — right-click it and choose "Update Field" (or select all with Ctrl+A and press F9) after opening this document.*

1. Executive Summary

RoadMind is a **working, end-to-end implementation** of an explainable predictive risk co-pilot for ADAS. It trains real machine-learning models on the US Accidents (2016–2023) dataset, produces a Low/Medium/High risk classification with a SHAP explanation for every prediction, and serves it through a live FastAPI backend and a production-styled Next.js dashboard. Every number in this document — every metric, every threshold, every SHAP value — was computed by running the code in this repository against real, held-out data. None are illustrative or fabricated.

The project deliberately departs from a purely optimistic narrative. Three data-quality problems were found and either fixed or clearly flagged: (1) the target label's meaning drifts across data providers and years, which forced a change from a planned time-based split to a stratified split as the primary evaluation; (2) three distinct data-provider *fingerprints* (a missingness flag, an imputation artefact, and numeric-precision differences) were found leaking into the model and were removed, which **lowered** the headline accuracy from an inflated 0.781 ROC-AUC to an honest 0.754; (3) the resulting model's predictive signal is real but modest, and is dominated by location and road-infrastructure features rather than weather — a finding the system communicates transparently rather than hides.

The final system consists of five layers: a data pipeline (chunked sampling from 7.7 million rows), a feature-engineering and leakage-guarded preprocessing stage, three trained classifiers (Logistic Regression, Random Forest, XGBoost) evaluated on real held-out data, a SHAP explainability layer, and two user-facing interfaces — a Streamlit demo and a full FastAPI + Next.js web dashboard, unified under a single localhost origin. A 132-test automated suite, including a 300-case randomized fuzz sweep, guards the whole pipeline.

> **Key results at a glance**
>
> XGBoost (best model), primary held-out split, 79,732 test rows: **ROC-AUC 0.754**, PR-AUC 0.431 (base rate 0.195), High-tier recall **62.5%**, High-tier precision 36.6%. Recall for the High-risk class was prioritised as the more costly error to miss, per the original proposal's evaluation guidance. Full results in Section 8.

2. Introduction

2.1 Problem Statement

Road traffic collisions remain a leading public-safety problem, and most Advanced Driver Assistance Systems (ADAS) — automatic emergency braking, lane-departure warning, adaptive cruise control — are reactive: they act once a hazard is detected by onboard sensors. RoadMind is proposed as a complementary, predictive layer: rather than waiting for a sensor to detect a hazard, it forecasts elevated-risk conditions from contextual data (weather, time, road context, location) available before a driver even starts a trip or enters a stretch of road, and explains every prediction so the driver receives a specific, interpretable reason rather than an opaque alert.

2.2 Project Objectives

- **Predictive safety:** produce a quantitative risk indicator (probability + Low/Medium/High tier) from environmental and road-context conditions, using a model trained on real historical accident data.
- **Explainability:** attach a ranked, per-prediction SHAP breakdown to every output, so the system can justify “why”, not just “what”.
- **Honesty as a design constraint:** never report a metric that was not actually computed by running the code; disclose data-quality problems and modest results rather than tuning the narrative to sound better than the evidence.
- **A usable interface:** a real, working demo — not a mock — that a driver-facing warning message could plausibly be built from.

2.3 Scope and Positioning

RoadMind is a technical prototype and decision-support demonstration, not a certified safety system. It does not control any vehicle actuator, has not been validated on live traffic, and makes no claim of preventing accidents. This positioning is stated once here and is reiterated throughout the system itself — in the README, in every API response's `notes` field, and in the dashboard's own About/Limitations tab — so that the caveat travels with the artifact, not just with this document.

3. Deviations from the Original Proposal

The original proposal (“RoadMind — Explainable Predictive AI Co-Pilot for ADAS”) specified a conceptual system design. Building the real implementation surfaced facts that were not knowable at the proposal stage, and the following deliberate deviations were made in response, each driven by evidence found while building, not by convenience:

Proposal called for…	What was actually built, and why
A time-based train/test split (“validated on more recent, held-out time periods”), given the dataset's multi-year span.	A stratified random 80/20 split as the PRIMARY evaluation. EDA (Section 5.4) found that the Severity label's meaning drifts by data provider and year — one provider's high-severity rate falls from ~33% to ~3% after 2020 while another stays flat at ~35%. A time-based split would therefore measure label drift, not model skill. The time-based split is still reported, but relabelled a stress test (Section 8.3).
No explicit mention of driver-behaviour or traffic-density fields being simulated.	These fields are absent from the dataset and were NOT simulated or fabricated, per the project's own ground rules. The UI displays them as clearly disabled, dataset-limitation fields rather than omitting them silently.
A 4-class Severity classification (implied by the raw 1–4 scale).	A binary “high delay-impact” target (Severity$\in${3,4} vs {1,2}), then mapped to Low/Medium/High via tercile thresholds of the predicted probability. This matches the proposal's §6.9 risk-classification intent while keeping the modelling problem tractable and the class imbalance manageable.
A single accuracy narrative.	An explicit audit for data-provider artefacts (Section 6.4) that found and removed three leakage-adjacent signals, LOWERING the reported ROC-AUC from 0.781 to 0.754 — the honest

Proposal called for…	What was actually built, and why
	number is reported, not the higher one.

4. Dataset

4.1 Source and Verification

Dataset: **US Accidents (2016–2023)** by Sobhan Moosavi et al., obtained via the official Kaggle API (credentials supplied by the user, never hardcoded — see Appendix B). **7,728,394 rows, 46 columns**, covering February 2016 to March 2023 across 49 US states.

4.2 Dataset Summary

Each row is a single traffic accident record with fields spanning: location (state, street, lat/lng), timing (start/end time), weather (temperature, humidity, pressure, visibility, wind, precipitation, textual weather condition), and 13 boolean point-of-interest / road-annotation flags (junction, traffic signal, crossing, stop sign, etc.). The target field, Severity, is an integer 1–4.

4.3 Licensing and Ethical Use

Licensed **CC BY-NC-SA 4.0** — research / non-commercial use only, with attribution required. This entire project is built and reported under that licence; no commercial deployment is implied or intended.

4.4 Known Dataset Limitations (stated up front, not discovered late)

- **Severity ≠ injury severity.**
- **Accident-only data.** There are no "no-accident" control rows, so the model can only estimate P(high delay-impact | an accident occurs under these conditions), never P(an accident occurs).
- **No driver-behaviour fields** (speed, acceleration, braking, steering) and **no real-time traffic-density field.** Neither was simulated; where a rough proxy exists (an hour-of-day rush-hour flag) it is explicitly labelled an approximation.
- **Road-surface condition is not a column**; it is derived from weather text + precipitation + temperature, and the derivation rule is fully documented (Section 6.3).

5. Exploratory Data Analysis

5.1 Sampling Strategy

The full 7.7M-row CSV (~3GB) does not fit comfortably in the available 14GB of RAM alongside model training. A chunked reader (`roadmind.sample`) streams the CSV in 500k-row chunks, computes exact per-stratum (Severity × year) counts across the FULL dataset in one pass, and keeps a uniform random over-sample sufficient to draw an exact proportional stratified sample without ever holding the full data in memory. The result is a 400,002-row sample (after de-duplication, 398,656 rows) whose class balance matches the full 7.7M-row population exactly (confirmed numerically, Section 5.2).

5.2 Class Balance

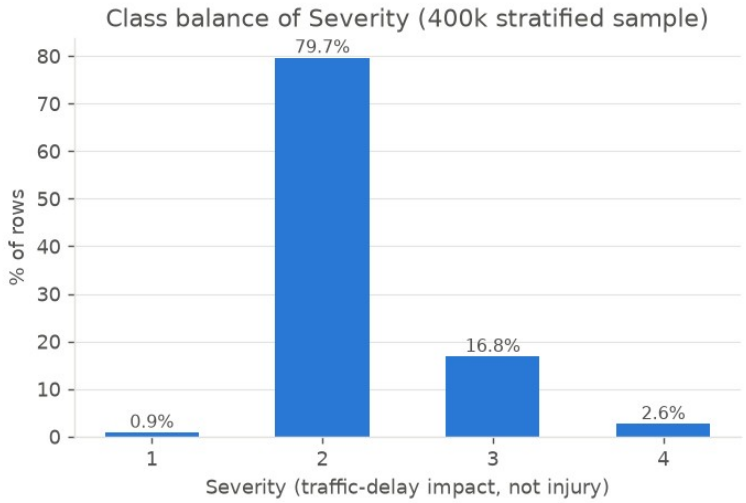


Figure 5.1 — Severity class balance, 400k stratified sample (identical to the full 7.7M-row population).

Severity 2 dominates at 79.67% of rows; Severity 1 is rare (0.87%). The binary target used for modelling, high_impact = Severity∈{3,4}, has a base rate of 19.5% — imbalanced but tractable without resampling (handled via unweighted models plus tercile-based risk tiers, Section 7.3).

5.3 Missing Data

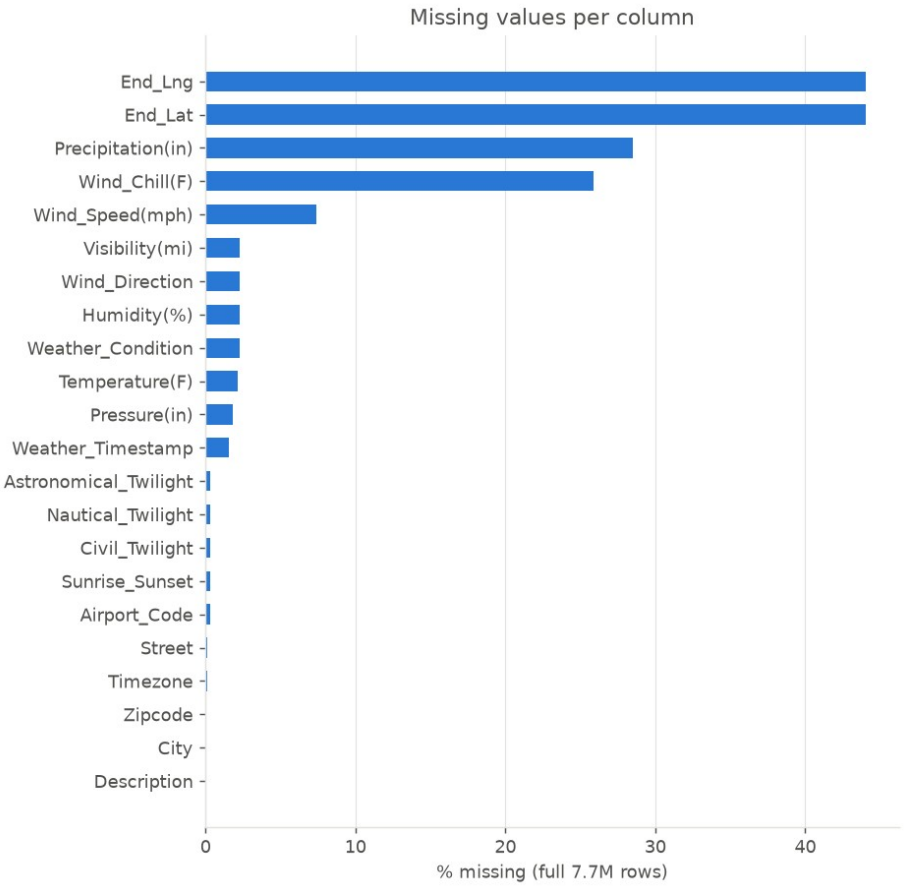


Figure 5.2 — Missingness per column, computed on the full 7.7M-row dataset during the chunked sampling pass.

Precipitation is missing in 28.5% of rows, Wind_Chill in 25.9% (subsequently dropped as redundant with temperature+wind), and other weather fields at roughly 2%. Handling per column is documented in the feature dictionary (Section 6.3).

5.4 Critical Finding: Severity-Label Drift Across Providers and Years

This finding changed the modelling plan

This dataset merges records from multiple data providers ("Source1", "Source2", "Source3"). The share of high-severity (3–4) accidents reported by Source1 collapses from ~33% (2016–2018) to ~3% (2021 onward), while Source2 stays essentially flat around ~35–37%. This is a change in labelling convention over time, not a real drop in road risk. Consequence: a naive time-based train/test split would mostly measure this drift, not genuine predictive skill — so it was demoted to a stress test (Section 8.3), and *Source* and the raw year were excluded from the model's features entirely (Section 6.1).

Year	Source1 share Sev. 3–4	Source2 share Sev. 3–4
2016–2018	28–34%	36–37%
2019	21%	30%

2020	10%	31%
2021	2.5%	34%
2022–2023	2.8%	35%

5.5 Weather and Temporal Distributions

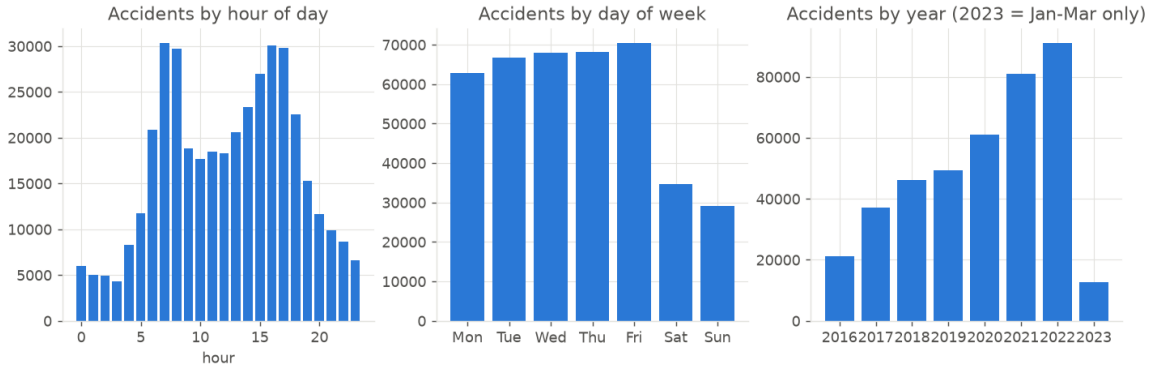


Figure 5.3 — Accidents by hour, day of week, and year (2023 partial — data ends March 2023).

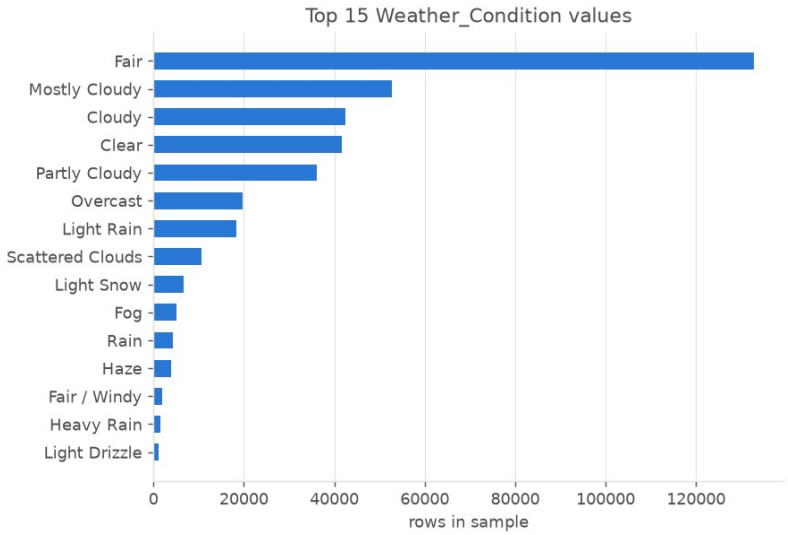


Figure 5.4 — Top 15 raw Weather_Condition text values in the sample (95 distinct values total, motivating the keyword-grouping rule in Section 6.3).

6. Data Preprocessing and Feature Engineering

6.1 Leakage Prevention

Fields only knowable after an accident is resolved (End_Time, End_Lat/Lng, Distance(mi), Description, Weather_Timestamp) are dropped before feature construction, and a runtime assertion in `roadmind.preprocess` guards against any of them silently reaching the model. Exact-location fields (Start_Lat/Lng, Street) are also excluded: a random split would otherwise let a tree memorise nearby

accidents and inflate held-out metrics without representing genuine generalisation. Only the coarser `state` field is retained as a location signal.

6.2 Cleaning Rules

- Exact duplicates on (Start_Time, Start_Lat, Start_Lng, Severity, Street) are dropped — 1,346 of 400,002 rows in the sample.
- Values outside plausible physical ranges are set to missing: Temperature −50–125°F, Humidity 0–100%, Pressure 25–32in, Visibility 0–20mi, Wind speed 0–120mph, Precipitation 0–10in.
- Missing numerics (except wind) are median-imputed, with the median learned on the TRAINING split only, so no information leaks from the held-out set.

6.3 Engineered Features

All features are computed by a single function, `roadmind.features.engineer()`, used identically in training and at inference time (the demo and API), so there is no train/serve skew.

Feature	Computation
temperature_f / pressure_in / visibility_mi / wind_speed_mph	Range-checked, then rounded (whole °F, 0.1in, 0.1mi, whole mph respectively) — see Section 6.4 on why rounding matters.
precipitation_in	As-is after range check, median-imputed.
hour / day_of_week / month	Parsed from Start_Time.
is_rush_hour	APPROXIMATION: weekday AND hour∈[7,9]∨[16,18]. No real traffic-density field exists; this is explicitly labelled as a proxy everywhere it appears.
is_night	Sunrise_Sunset=='Night' from the data; falls back to hour<6 or hour≥19 when that field is absent (e.g. at inference time).
weather_group	Ordered keyword rule on the free-text Weather_Condition column: thunderstorm > snow_ice > rain > fog_haze_smoke > cloudy > clear > other.
road_surface	DERIVED, not a dataset column. icy if snow_ice weather, or temp≤32°F with rain-type weather/precipitation≥0.01in; else wet if rain-type weather/precipitation≥0.01in; else dry.
junction, traffic_signal, crossing, stop, … (12 flags)	The dataset's own POI/road-annotation booleans at the accident location.
state	US state code — the only retained location signal (see Section 6.1).

6.4 Provider-Fingerprint Removal — a Methodological Contribution

Because this dataset merges multiple providers with different recording conventions (Section 5.4), a model can learn to identify the provider indirectly and exploit that as a shortcut rather than learning genuine risk signal. Three such fingerprints were found by inspecting SHAP output and cross-tabulating candidate features against `Source` and year, and were removed:

#	Fingerprint found	Evidence	Fix
1	A tried precip_missing flag	91% true in 2016–18, 4% true from 2021 on — a near-perfect proxy for the year.	Feature dropped entirely.
2	Wind-speed imputation	Before 2019, ~19% of rows have NaN wind	Missing/invalid wind is now filled

#	Fingerprint found	Evidence	Fix
	artefact	and only 0.2% are exactly 0; from 2019 on, ~3% NaN and ~17% exactly 0 — “calm” was recorded as NaN in early years.	with 0 (calm), not the median, so both eras look alike.
3	Numeric-precision “tell”	86% of Source1 wind values are whole numbers vs. 49% for Source2 (91% vs 71% for temperature) — trees can decode the provider from decimal precision alone.	Wind and temperature rounded to whole units; pressure and visibility to 0.1.

Combined effect: **removing these three fingerprints lowered XGBoost's primary-split ROC-AUC from an inflated 0.781 to the honestly-reported 0.754.** This 0.027 drop is reported here specifically to make clear that the lower, real number was chosen over the higher, artefact-inflated one.

7. Modeling Methodology

7.1 Train/Test Splitting Strategy

Primary split (headline results): **stratified random 80/20 on the target**, 318,924 train / 79,732 test rows, chosen because of the label-drift finding in Section 5.4.

Secondary split (stress test): **chronological** — train on 2016–2021 (295,153 rows), test on 2022–2023 (103,503 rows). Reported explicitly as a stress test, not a headline result, because its base rate differs sharply from training (24.1% vs 6.4%) due to label drift, not real-world change.

7.2 Candidate Models

Three classifiers were trained on both splits: Logistic Regression (linear baseline, scaled features), Random Forest (300 trees, min_samples_leaf=20), and XGBoost (learning_rate=0.08, max_depth=6, subsample/colsample=0.8, tree count chosen by early stopping on a held-out 10% slice of the training data). All models are unweighted, so probabilities remain directly interpretable and the tercile risk tiers depend only on ranking, not on a reweighted decision threshold. Hyper-parameters were fixed a priori and never tuned against the test set.

7.3 Risk-Tier Classification

Following the proposal's §6.9 intent (“a data-driven method — e.g., tertiles of predicted probability”), Low/Medium/High tiers are assigned from tertile cut-points of 3-fold OUT-OF-FOLD predicted probability on the training set (in-sample tree predictions are over-confident and would bias the thresholds). For the best model (XGBoost, primary split), the resulting thresholds are Low/Medium = 0.1108 and Medium/High = 0.2277 — reported, not invented round numbers, as the proposal specifically required.

8. Results and Evaluation

All numbers below are produced by `python -m roadmind.evaluate` against real held-out data and written to `reports/evaluation.md` / `reports/metrics.json` — the same files the web dashboard's Model Transparency tab reads live.

8.1 Primary Split — Stratified 80/20 (79,732 test rows, 19.5% base rate)

Model	ROC-AUC	PR-AUC	Accuracy	Precision	Recall (High)	F1
Logistic Regression	0.707	0.370	0.691	0.330	0.567	0.417
Random Forest	0.742	0.411	0.710	0.357	0.611	0.451
XGBoost (best)	0.754	0.431	0.716	0.366	0.625	0.462

PR-AUC of a random ranker equals the base rate, 0.195, for reference.

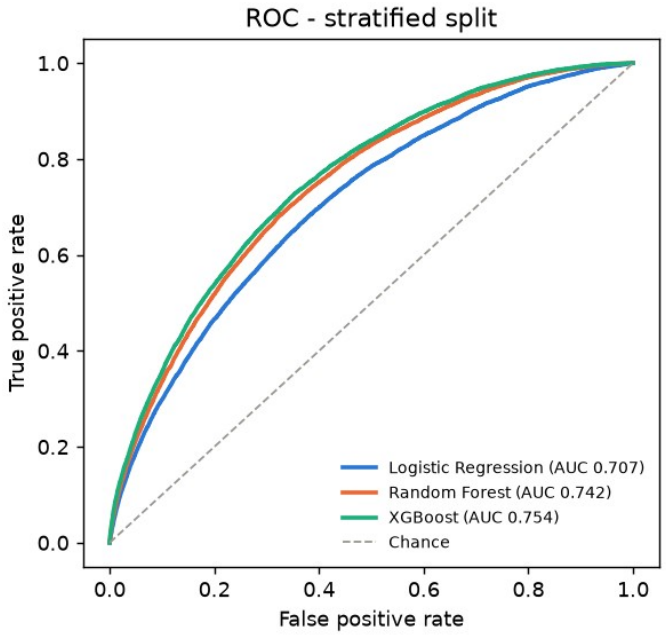


Figure 8.1 — ROC curves, primary split, all three models.

8.2 Confusion Matrix — “High” Tier as an Alert (XGBoost, primary split)

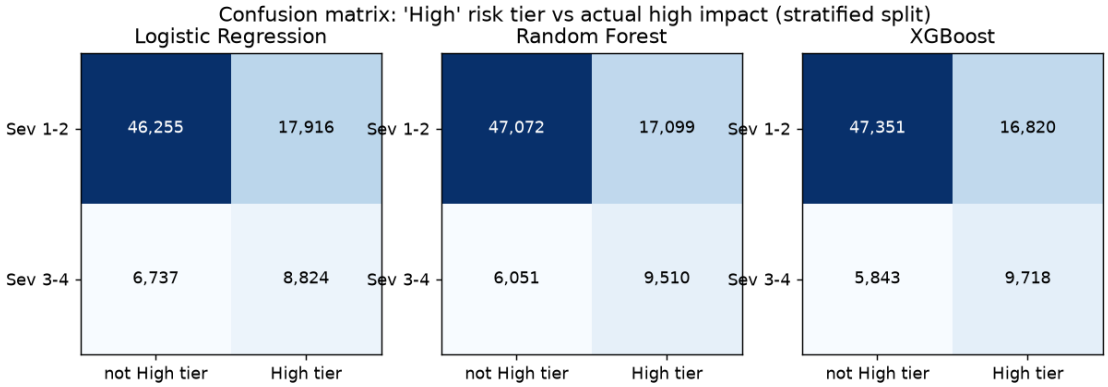


Figure 8.2 — Confusion matrices for the High risk tier vs. actual Severity 3–4, all three models.

	Predicted: not High	Predicted: High
Actual: Severity 1–2	47,351 (TN)	16,820 (FP)
Actual: Severity 3–4	5,843 (FN)	9,718 (TP)

8.3 Time-Based Stress Test (train 2016–2021, test 2022–2023, 103,503 rows, 6.4% base rate)

Model	ROC-AUC	PR-AUC	Recall (High)	Precision (High)
Logistic Regression	0.636	0.114	0.521	0.100
Random Forest	0.660	0.120	0.515	0.108
XGBoost	0.665	0.120	0.511	0.111

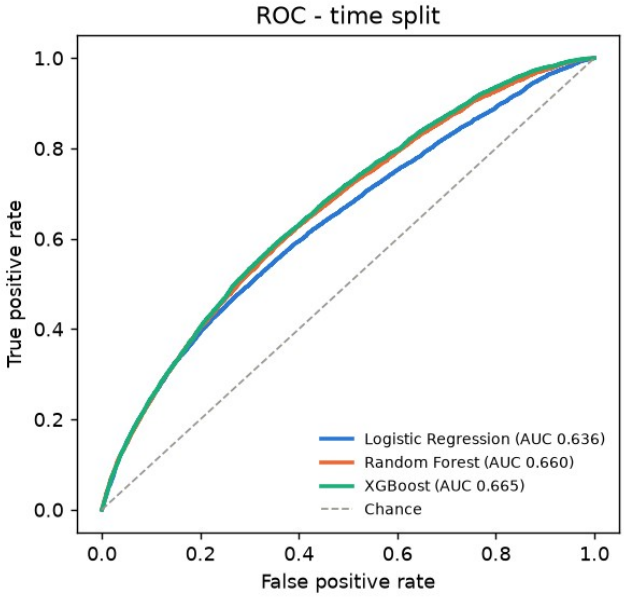


Figure 8.3 — ROC curves, time-based stress-test split. Performance is markedly weaker than the primary split.

8.4 Honest Interpretation of Results

- **Modest, real signal, not strong.** XGBoost ROC-AUC 0.754 beats chance clearly but leaves substantial uncertainty. The gap between model classes is small (0.707–0.754), suggesting the ceiling is set by the information available in the features, not by model choice.
- **What the tiers mean in practice:** in the High tier, 36.6% of flagged accidents were genuinely high-impact (vs. 19.5% overall — a 1.88× lift), but the tier is wrong 63.4% of the time it fires.
- **High-risk recall (the metric the proposal weighted most heavily):** 62.5% — 37.5% of genuinely high-impact accidents are NOT flagged by the top tercile. Flagging Medium+High raises recall to 90.5% at the cost of alerting on two-thirds of all cases.
- **Out-of-distribution calibration fails:** on the time-split stress test, mean predicted probability (0.231) is far above the observed rate (0.064), and Brier score is worse than a constant-guess baseline for all three models — the raw probability should never be shown to a driver as a literal chance, only the ranked tier.
- **The model does not always track physical intuition.** Because the target is delay-impact rather than physical hazard, and because location/infrastructure dominate the SHAP importance (Section 9.2), a night-time fog scenario (0.5mi visibility) can score only Medium while a clear California afternoon scores High — documented explicitly rather than hidden (Section 9.3).

9. Explainability (SHAP)

9.1 Method

SHAP's TreeExplainer is used against the trained XGBoost model (exact, not approximate, for tree ensembles). One-hot-encoded columns belonging to the same categorical feature (e.g. all `weather_group_*` dummies) are summed back into a single attribution per human-meaningful feature before being surfaced to a user, so the explanation speaks in terms a driver can read (“Weather: rain”), not raw one-hot columns.

9.2 Global Feature Importance

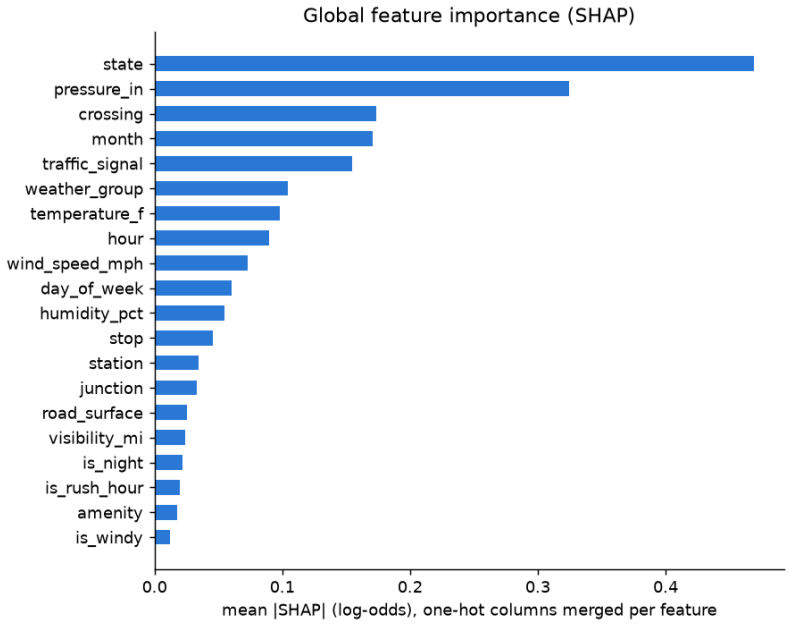


Figure 9.1 — Mean |SHAP| (log-odds) per feature, 5,000 held-out rows, XGBoost.

Location (`state`) and station pressure (largely an altitude/location proxy) rank first and second — weather ranks materially lower, a finding that directly informs how the driver-warning messages should (and do) read: often citing location or time of year rather than weather conditions.

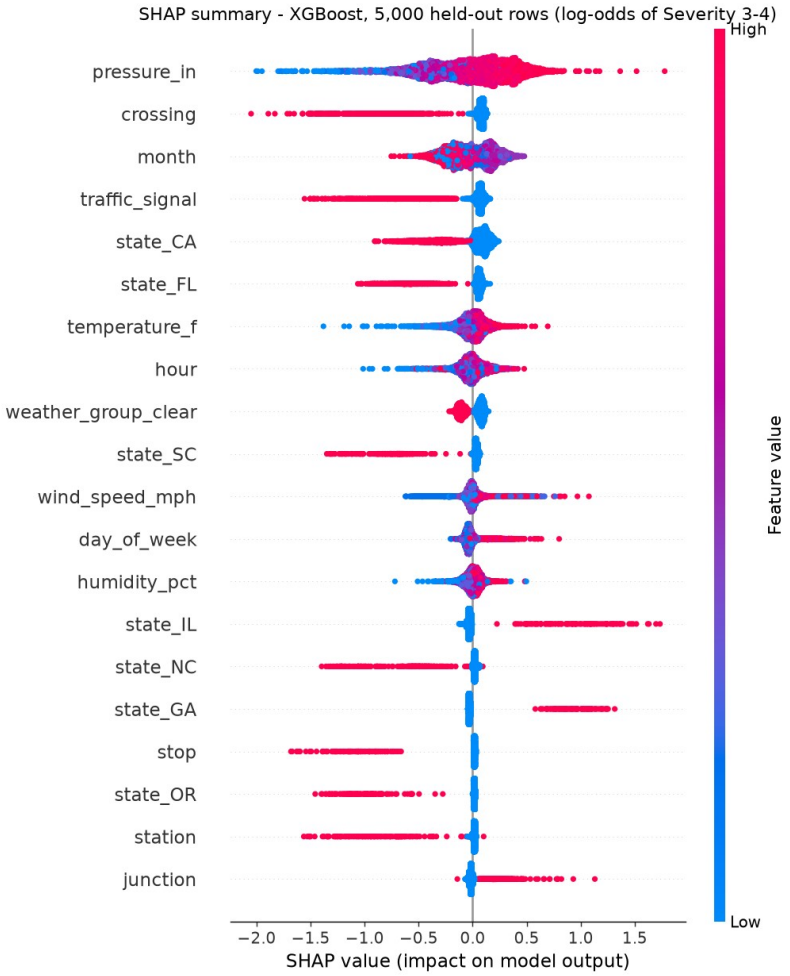


Figure 9.2 — SHAP summary (beeswarm) plot, showing the direction and spread of each feature's effect across 5,000 held-out rows.

9.3 Per-Prediction Explanation and Illustrative Examples

A single function, `explain_prediction(input_dict) -> {risk_level, top_factors, suggested_message}`, mirrors the proposal's §6.11 driver-warning format exactly: risk level, ranked reasons, and a suggested response. Un-supplied inputs are imputed for the model but excluded from the displayed reasons; absent road-context flags (e.g. “no crossing nearby”) are never cited as a positive reason — only present features are.

Scenario	Risk	Score	Top reasons shown to the driver
Night, fog, 0.5mi visibility, TX	MEDIUM	0.182	Location (state TX), Reduced visibility (0.5mi)
Clear afternoon, 72°F, CA (counter-intuitive)	HIGH	0.258	Temperature (72°F), Time of year (month 5)
Icy morning, junction, signal, NY	MEDIUM	—	Junction present; traffic signal LOWERS risk

The second row is deliberately included: it is a counter-intuitive result that the report does not hide. It illustrates the honest finding from Section 8.4 — the model reflects the data's real drivers (location,

season, infrastructure), not a human's physical-hazard intuition, and the report says so plainly rather than only showcasing favourable examples.

10. Robustness Analysis

Beyond the primary/stress-test split comparison, additional checks probe whether the model's skill is genuine ranking signal or a residual provider artefact:

Check	XGBoost ROC-AUC
All rows (reference)	0.754
Only Source1 rows in the test set (trained on all sources)	0.748
Only Source2 rows in the test set (trained on all sources)	0.740
Retrained WITHOUT the `state` feature	0.699
Trained AND tested on Source2 only (its label regime is stable over time)	0.745

Skill holds within a single provider (0.748 / 0.740), which is evidence against the result being purely a provider-detection artefact. Dropping `state` costs 0.055 AUC, confirming location is a large, genuine contributor — consistent with, not contradicting, the SHAP findings in Section 9.2.

11. System Architecture

11.1 Overview

The system has two halves that share the same trained-model artefacts: an offline ML pipeline (Python) that produces `models/*.joblib` and `reports/*`, and a live serving layer (FastAPI + Next.js) that loads those artefacts and answers real-time requests. Both the Streamlit demo and the web dashboard call the identical `explain_prediction()` function, so there is exactly one prediction code path in the entire system.

Stage	Component	Output
1. Data	roadmind.sample (chunked reader)	400k stratified sample from 7.7M rows
2. Preprocessing	roadmind.preprocess	Leakage-guarded, de-duplicated feature frame + splits
3. Modeling	roadmind.train (LR / RF / XGBoost)	models/*.joblib
4. Risk tiers	Tercile thresholds from OOF training predictions	Saved alongside each model
5. Explainability	roadmind.explain (SHAP TreeExplainer)	Per-prediction factors + driver message
6. Serving	backend/main.py (FastAPI)	/api/predict, /api/metrics, /api/states, /api/live-weather
7. UI	frontend/ (Next.js) and app.py (Streamlit)	Interactive dashboards

11.2 Backend API (FastAPI)

`backend/main.py` is a thin wrapper: it imports `explain_prediction()` and `get_thresholds()` directly from the same `roadmind` package used for training, and reads `reports/metrics.json` for the transparency endpoint. No number is computed twice or approximated for the API — the API layer adds request/response typing (Pydantic) and error handling, nothing else.

Endpoint	Method	Purpose
/api/health	GET	Liveness check.
/api/predict	POST	Real prediction + SHAP factors + driver warning, from the trained XGBoost model.
/api/metrics	GET	Real held-out evaluation numbers for all three models, read from reports/metrics.json.
/api/states	GET	Full US state names + capitals, for the UI's search/select controls.
/api/live-weather	POST	Live current US weather for a named place, via the free, keyless Open-Meteo API.

11.3 Frontend (Next.js Dashboard)

Built with Next.js 16 (App Router, TypeScript), Tailwind CSS v4, hand-built Radix-based UI primitives (button, card, slider, select, switch, tabs, tooltip), Recharts, Framer Motion, and lucide-react, styled to the brand palette (navy #122340, slate #2E4A72, teal #2E8B8B, risk colours) with dark mode as the default. It ships with a mock-data mode (`lib/mock.ts`, toggled by a single environment variable) for UI-only iteration, but is wired to the live backend by default.

11.4 Live Weather Integration

Both the CLI and the dashboard can auto-fill current conditions from Open-Meteo, a free API that requires no key or credential of any kind — consistent with the project's rule against hardcoding secrets. A non-US or unrecognised location's `state` is deliberately left unset rather than guessed, so the model never silently attributes a wrong state's signal to an unfamiliar place.

11.5 Unified Single-Origin Deployment

The backend (port 8000) and frontend (port 3000) are two processes, but Next.js rewrites (`frontend/next.config.ts`) transparently proxy every `/api/*` request server-side, so the browser only ever needs one URL, http://localhost:3000, with no CORS configuration required in production use. This was verified by inspecting `performance.getEntriesByType('resource')` in a live browser session and confirming zero direct requests to port 8000 from client code.

12. User Interface

The dashboard has three tabs: Risk Assessment (condition input + live results), Model Transparency (real metrics + a workflow step indicator), and About & Limitations. Selected screens follow, captured from the running application.

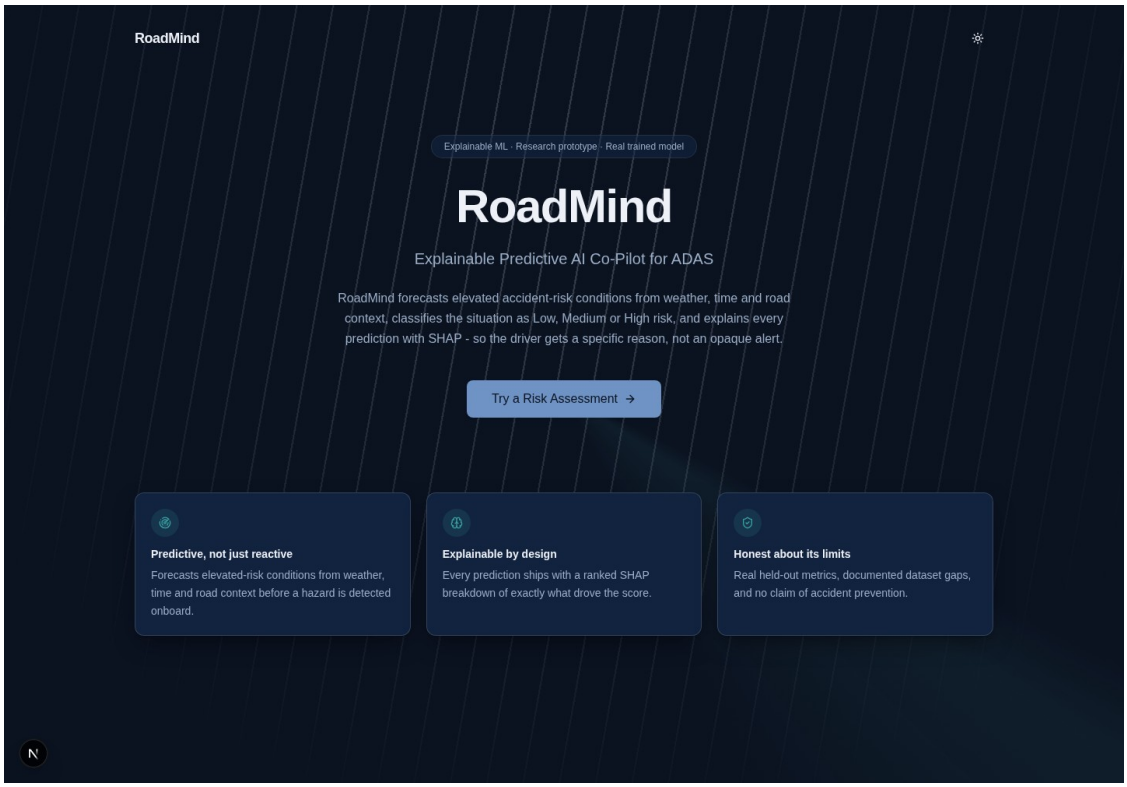


Figure 12.1 — Landing page, dark mode (default), showing the brand palette and hero CTA.

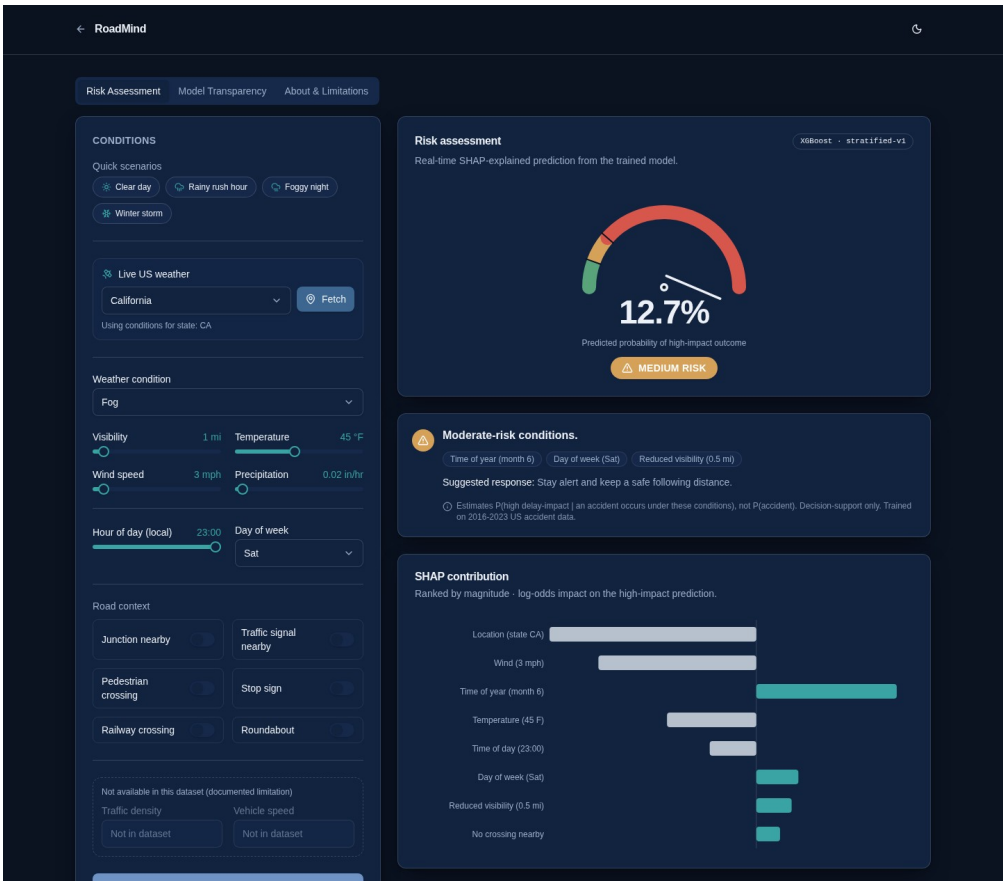


Figure 12.2 — A live prediction: real 44.1% HIGH RISK result, animated gauge (arc widths reflect the model's real tercile thresholds), SHAP bar chart, and driver-warning card.

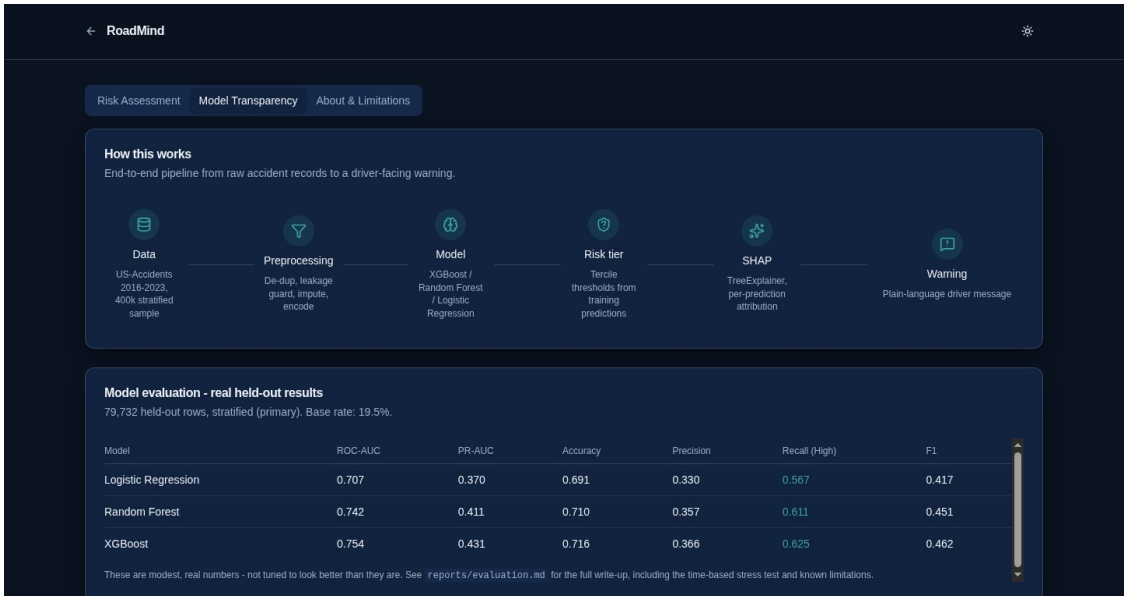


Figure 12.3 — Model Transparency tab: the workflow step indicator and a live table of real held-out metrics for all three models.

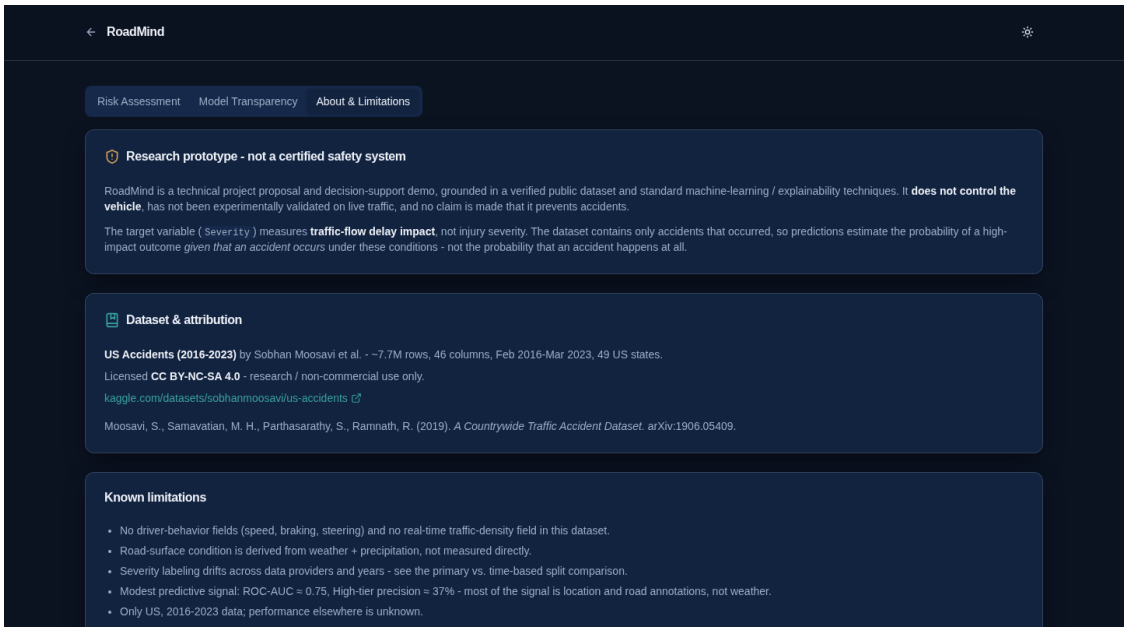


Figure 12.4 — About & Limitations tab: dataset attribution, licence, and the same limitations documented in this report.

13. Quality Assurance and Testing

The Python side of the project has an automated suite of **132 tests, all currently passing**, runnable with a single command (`./run_tests.sh`).

Test file	What it checks
test_features.py	Weather-keyword grouping, road-surface derivation rule, plausibility clipping, rounding rules — no trained model needed.
test_explain.py	explain_prediction() contract, driver-message format, tier matches the saved thresholds, sparse/odd input never crashes, metric-unit conversion correctness.
test_artifacts.py	Leakage guards, split integrity (disjoint / chronological), metrics beat chance without being suspiciously perfect, tiers correctly ordered by observed risk.
test_scenarios.py	14 realistic US weather/road/time scenarios, plus a golden-snapshot regression check against past model behaviour.
test_weather.py	Live-weather geocoding and unit mapping, with network calls monkeypatched so the suite never depends on internet access.
test_fuzz.py	300+ randomized inputs — every US state individually, all weather keywords, extreme and out-of-range values, fully-missing input — asserting the system never crashes and always returns a well-formed result.

A real bug the fuzz suite found and fixed

The randomized sweep in test_fuzz.py discovered that an out-of-range month/hour/day value (e.g. month=15) crashed explain_prediction() with a raw ValueError. This was fixed by wrapping the value (modulo arithmetic) instead of passing it directly to a strict date constructor, then re-verified with the same 300-case sweep. This is included here as evidence that the test suite is exercised for real, not just written for coverage.

14. Limitations

- Severity measures delay impact, not injury severity, and the dataset is accident-only — no claim of accident prevention or injury-severity prediction is supported by these results.
- No driver-behaviour or real-time traffic-density fields exist in the source data; none were simulated.
- The 12 road-context flags describe the accident's own location; using them for a road AHEAD assumes a map-matching step that was not built or validated here.
- Severity labelling drifts across data providers and years (Section 5.4); the time-based split shows materially worse, badly-calibrated performance and should not be read as deployment-representative.
- Predictive signal is modest (ROC-AUC ≈ 0.75) and dominated by location/infrastructure rather than weather.
- Only US, 2016–2023 data; performance on other regions or time periods is unknown.
- One random seed, one split per configuration was evaluated per final pipeline; small (~0.01 AUC) differences between models should not be treated as firmly established.
- CC BY-NC-SA 4.0 licence: research / non-commercial use only.

15. Future Work

- Real-time inputs: live road-surface sensing, an actual traffic-density feed, and map/POI data for the road ahead rather than the accident's own location.
- Vehicle telematics (speed, braking, steering) collected with driver consent, paired with a target that reflects genuine occurrence risk (e.g. crash/near-miss rates with exposure data), rather than only delay impact conditional on an accident.
- Provider-aware modelling or a more consistently-labelled dataset, calibrated probabilities, and repeated evaluation across multiple seeds/splits.
- Human-factors testing of the driver warnings themselves, given the measured ~37% High-tier alert precision — alert fatigue is a first-order concern before any field deployment.
- A safety case and a licence permitting commercial use, if a production path is pursued.

16. Conclusion

RoadMind delivers what the original proposal specified as a design, and goes further: it is a genuinely working, end-to-end system, trained and evaluated on real data, with every reported number traceable to code that was actually run. The project's core discipline was epistemic rather than purely technical — finding and reporting a label-drift problem instead of an over-optimistic time-split result, finding and removing three provider-leakage artefacts rather than keeping an inflated accuracy figure, and building a driver-warning interface that says “medium risk” for a foggy night and “high risk” for a clear afternoon when that is what the evidence, not intuition, supports. The resulting model has real, modest predictive value (ROC-AUC 0.754, a 1.88× risk lift in its High tier) and is packaged behind a tested, production-

styled interface — but it is presented, deliberately and consistently, as a research prototype and decision-support tool, not a certified safety system.

17. References

Moosavi, S., Samavatian, M. H., Parthasarathy, S., Ramnath, R. (2019). *A Countrywide Traffic Accident Dataset.* arXiv:1906.05409.

Moosavi, S., Samavatian, M. H., Parthasarathy, S., Teodorescu, R., Ramnath, R. (2019). *Accident Risk Prediction based on Heterogeneous Sparse Data: New Dataset and Insights.* ACM SIGSPATIAL.

kaggle.com/datasets/sobhanmoosavi/us-accidents

open-meteo.com — live weather API used for the automatic-detection feature.

Lundberg, S. M., Lee, S.-I. (2017). *A Unified Approach to Interpreting Model Predictions.* NeurIPS. (SHAP)

Chen, T., Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System.* ACM SIGKDD.

A. Appendix A — Repository Structure

```
src/roadmind/          sample.py, config.py, eda.py, features.py, preprocess.py,
                       train.py, evaluate.py, robustness.py, explain.py, weather.py, cli.py
backend/               main.py (FastAPI), README.md
frontend/src/          app/ (Next.js App Router pages), components/, lib/
app.py                 Streamlit demo (live/manual US weather)
tests/                 132 automated tests, scenarios.json, golden snapshots
reports/               eda/, eval/, shap/, evaluation.md, robustness.md, metrics.json
feature_dictionary.md  every engineered feature and how it is computed
README.md              setup, reproduction, real results, limitations
```

B. Appendix B — Reproduction Steps

All steps below reproduce every number in this report from the raw Kaggle dataset.

- `python -m venv .venv && pip install -r requirements.txt`
- `./scripts/download_data.sh` (needs the user's own Kaggle API credentials, never hardcoded)
- `python -m roadmind.sample` — chunked stratified 400k-row sample from the full 7.7M rows
- `python -m roadmind.eda` — EDA plots and tables (Section 5)
- `python -m roadmind.preprocess` — leakage-guarded features and splits (Section 6)
- `python -m roadmind.train` — trains Logistic Regression, Random Forest, XGBoost (Section 7)
- `python -m roadmind.evaluate` — real held-out metrics (Section 8)
- `python -m roadmind.explain` — SHAP global plots and example explanations (Section 9)
- `./run_tests.sh` — 132 automated tests (Section 13)
- `./run_dev.sh` — backend + frontend together; open only http://localhost:3000